

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

«МИРЭА - Российский технологический университет»

Институт перспективных технологий и индустриального программирования

Кафедра индустриального программирования

ПРАКТИЧЕСКАЯ РАБОТА

Тема: «Симулятор работы лифтов в здании»

Дисциплина: «Программирование корпоративных систем»

Выполнила:

студентка группы ЭФБО-06-24

Кальченко Евгения Витальевна

Преподаватель:

Сокунов Дмитрий Антонович

Москва, 2026

1. Введение

Данный отчёт описывает учебный проект «Симулятор работы лифтов в здании». Проект представляет собой консольное приложение на C++17, моделирующее работу нескольких лифтов, обработку заявок с этажей, выбор лифта по стратегии диспетчеризации и вывод итоговой статистики.

Цель проекта - показать объектно-ориентированную структуру приложения, работу с CMake, unit-тестами и контейнеризацией через Docker. Проект не использует графический интерфейс и не подключается к реальному оборудованию. Все лифты, заявки и события моделируются программно.

Симуляция выполняется последовательно: программа принимает заявки, распределяет их между лифтами, обрабатывает очередь и выводит итоговую статистику. Основной акцент сделан на структуре классов, стратегиях выбора лифта, тестировании и запуске проекта в контейнере.

2. Общее описание программного продукта

Программный комплекс выступает учебной моделью работы группы лифтов в многоквартирном здании. Пользователь запускает программу и может передать количество этажей, количество лифтов и стратегию выбора лифта.

Основные функции системы:

- создание лифтов;
- создание заявок на этажи;
- постановка заявок в очередь диспетчера;
- выбор лифта по стратегии;
- назначение заявки выбранному лифту;
- обработка очереди заявок лифтом;
- сбор и вывод статистики.

Проект имеет плоскую структуру: папка `include` содержит заголовочные файлы, `src` содержит реализацию, `tests` содержит unit-тесты, а `tests/scenarios` содержит демонстрационные сценарии.

3. Спецификация требований

Функциональные требования проекта:

- приём заявок с указанием этажа и направления;
- назначение заявки конкретному лифту;
- поддержка двух стратегий диспетчеризации: ближайший лифт и минимальная очередь;
- последовательная обработка очереди заявок лифтом;
- вывод событий работы симулятора в консоль;
- сбор статистики по обработанным заявкам и лифтам;
- запуск unit-тестов и демонстрационных сценариев.

Нефункциональные требования:

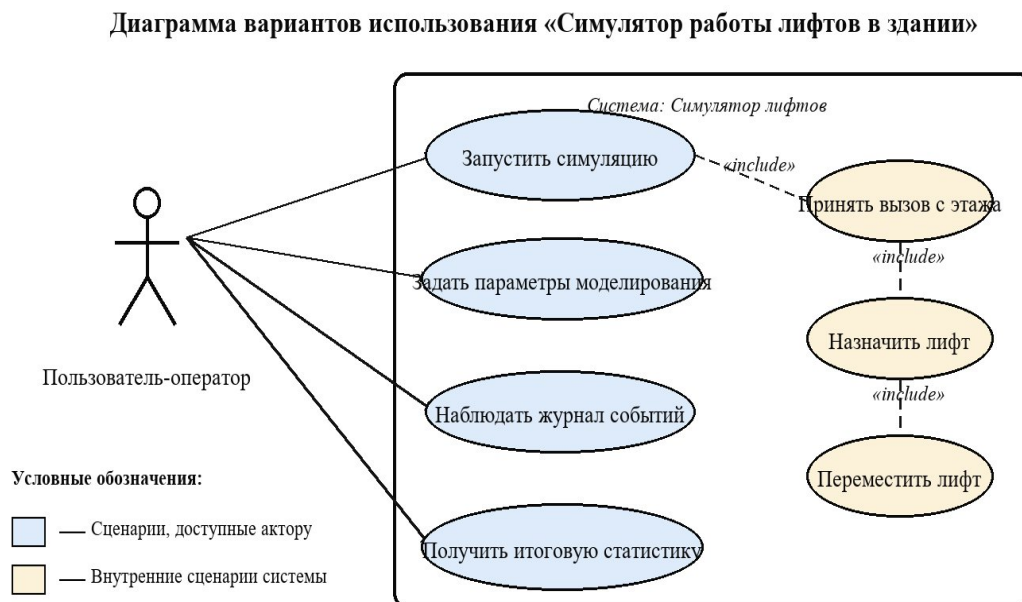
- язык реализации C++17;
- сборка через CMake;
- проверка через Catch2 и ctest;

- запуск в Docker-контейнере;
- простая последовательная обработка заявок.

4. Визуальное моделирование

4.1. Диаграмма вариантов использования

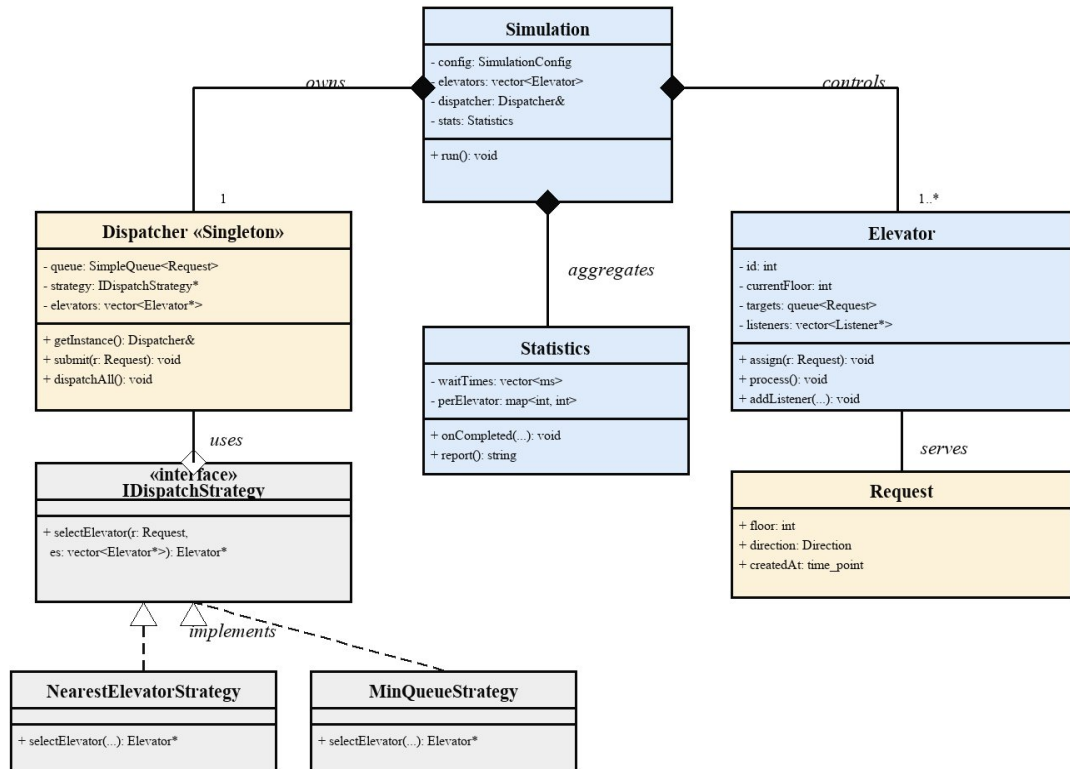
Диаграмма вариантов использования показывает, какие действия может выполнять пользователь при работе с проектом.



4.2. Диаграмма классов

Диаграмма классов показывает основные классы, интерфейсы и связи между ними. Dispatcher хранит очередь заявок и выбранную стратегию, стратегии реализуют общий интерфейс IDispatchStrategy, Elevator обрабатывает заявки, а Statistics получает уведомления о выполненных заявках.

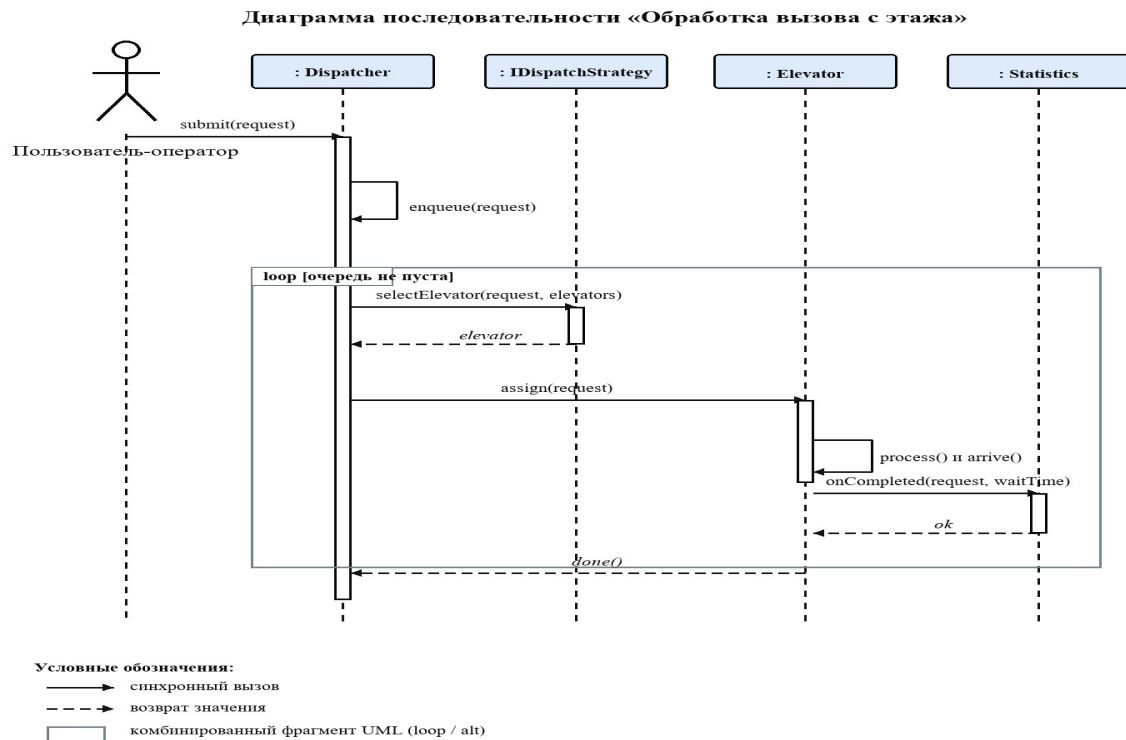
Диаграмма классов «Симулятор работы лифтов в здании»



Основные связи: Dispatcher хранит очередь заявок и выбранную стратегию; стратегии реализуют IDispatchStrategy; Elevator обрабатывает заявки; Statistics получает уведомления от Elevator через IRequestCompletionListener; Simulation объединяет все части проекта.

4.3. Диаграмма последовательности

Диаграмма последовательности показывает типичный сценарий запуска симуляции и обработки заявки.



Для реализации симулятора выбраны паттерны Singleton, Strategy, Factory Method и Observer. Они помогают разделить ответственность между классами: диспетчер принимает заявки, стратегии выбирают лифт, фабрика создаёт нужную стратегию, а статистика получает уведомления о выполненных заявках.

5. Паттерны проектирования

На основании требований к системе и спроектированной UML-архитектуры для реализации учебного симулятора лифтов были выбраны следующие паттерны проектирования.

5.1. Порождающий паттерн: Singleton

Singleton используется для класса Dispatcher. В системе должен быть один общий диспетчер, который принимает заявки и распределяет их между лифтами.

5.2. Поведенческий паттерн: Strategy

Strategy используется для выбора алгоритма диспетчеризации. Благодаря этому можно запускать симуляцию с разными стратегиями: выбор ближайшего лифта или выбор лифта с минимальной очередью.

5.3. Порождающий паттерн: Factory Method

Factory Method используется в StrategyFactory. Фабрика создаёт нужную стратегию по строковому названию, переданному при запуске программы.

5.4. Поведенческий паттерн: Observer

Observer используется для передачи событий от лифта к статистике. Лифт сообщает о завершении заявки, а Statistics сохраняет данные для итогового отчёта.

6. Детальное описание применения паттернов проектирования

В данном разделе подробно описано применение паттернов проектирования в классах проекта.

6.1. Singleton: класс Dispatcher

Dispatcher используется как единый объект диспетчера. Метод Dispatcher::getInstance() возвращает общий диспетчер, через который проходят заявки.

```
class Dispatcher {
public:
    static Dispatcher& getInstance();
    void setStrategy(std::unique_ptr<IDispatchStrategy> strategy);
    void registerElevators(std::vector<Elevator*> elevators);
    void submit(Request request);
    void dispatchAll();
private:
    Dispatcher() = default;
    SimpleQueue<Request> queue_;
    std::unique_ptr<IDispatchStrategy> strategy_;
    std::vector<Elevator*> elevators_;
};
```

Singleton нужен для того, чтобы в симуляции был один общий диспетчер, через который проходят заявки.

6.2. Strategy: интерфейс IDispatchStrategy

Алгоритм выбора лифта вынесен из Dispatcher в отдельные стратегии. Это позволяет запускать программу с разными правилами выбора лифта.

```
struct IDispatchStrategy {
    virtual ~IDispatchStrategy() = default;
    virtual Elevator* selectElevator(
        const Request& request,
        const std::vector<Elevator*>& elevators) = 0;
};

class NearestElevatorStrategy : public IDispatchStrategy { ... };
class MinQueueStrategy : public IDispatchStrategy { ... };
```

NearestElevatorStrategy выбирает ближайший лифт, а MinQueueStrategy выбирает лифт с самой короткой очередью.

6.3. Factory Method: фабрика стратегий

StrategyFactory создаёт стратегию по строковому названию. Это нужно, потому что пользователь передаёт стратегию как текст: nearest или min_queue.

```

std::unique_ptr<IDispatchStrategy>
StrategyFactory::create(const std::string& name) {
    if (name == "nearest") {
        return std::make_unique<NearestElevatorStrategy>();
    }
    if (name == "min_queue") {
        return std::make_unique<MinQueueStrategy>();
    }
    throw std::invalid_argument("Unknown dispatch strategy: " + name);
}

```

6.4. Observer: уведомление Statistics о событиях лифта

Паттерн Observer подходит для связи лифта и статистики: лифт завершает обработку заявки и уведомляет отдельный объект, который собирает результаты работы системы.

Лифт не считает статистику сам. Он только сообщает, что заявка выполнена. Класс Statistics подписывается на эти события через интерфейс IRequestCompletionListener.

```

struct IRequestCompletionListener {
    virtual ~IRequestCompletionListener() = default;
    virtual void onCompleted(
        const Request& request,
        std::chrono::milliseconds waitTime) = 0;
};

class Statistics : public IRequestCompletionListener {
public:
    void onCompleted(const Request& request,
        std::chrono::milliseconds waitTime) override;
    std::string report() const;
};

```

Simulation создаёт объект Statistics и передаёт его каждому лифту через addListener(). Когда Elevator обрабатывает заявку, он вызывает onCompleted(), и статистика сохраняет время ожидания.

6.5. Сводная таблица применения паттернов

Паттерн	Категория	Классы / роль в проекте
Singleton	Порождающий	Dispatcher - единая точка диспетчеризации
Strategy	Поведенческий	IDispatchStrategy, NearestElevatorStrategy, MinQueueStrategy
Factory Method	Порождающий	StrategyFactory::create создаёт стратегию по имени
Observer	Поведенческий	IRequestCompletionListener, Statistics получает события

		or Elevator
--	--	-------------